

const

一、const 的本质

const 是 C++ 中的**类型限定符 (type qualifier)**，表示**"只读/不可修改"**。编译器会在编译期检查是否存在违反 const 约束的操作。

```
const int x = 10;
x = 20; // ❌ 编译错误：不能修改 const 变量
```

二、const 修饰变量

2.1 const 修饰普通变量

```
const int a = 10; // 必须初始化
int const b = 20; // 与上面等价 (const 和类型名可交换位置)

a = 30; // ❌ 编译错误
```

与宏定义 #define 的对比：

特性	#define PI 3.14	const double PI = 3.14
时机	预处理阶段 文本替换	编译阶段类型检查
类型	无类型	有明确类型
作用域	全局（无作用域）	遵循正常作用域规则
调试	符号表中无名字	有名字，可调试
推荐	❌	✅

2.2 const 修饰数组

```
const int arr[] = {1, 2, 3};
arr[0] = 10; // ❌ 编译错误：元素不可修改
```

2.3 const 与编译期常量

```
const int SIZE = 100;
int arr[SIZE]; // ✅ C++ 中 const 整型可作为数组大小

// constexpr (C++11) 更明确表示编译期常量
```

```
constexpr int SIZE2 = 200;
int arr2[SIZE2];      // ✓
```

三、const 与指针

3.1 四种组合

```
int x = 10, y = 20;

// ① 指向常量的指针（底层 const）
const int* p1 = &x;
*p1 = 30;      // ✗ 不能修改所指内容
p1 = &y;        // ✓ 可以改变指向

// ② 常量指针（顶层 const）
int* const p2 = &x;
*p2 = 30;      // ✓ 可以修改所指内容
p2 = &y;        // ✗ 不能改变指向

// ③ 指向常量的常量指针
const int* const p3 = &x;
*p3 = 30;      // ✗ 不能修改所指内容
p3 = &y;        // ✗ 不能改变指向
```

3.2 const 指针的赋值兼容性

```
int x = 10;
const int cx = 20;

// 非 const → const: ✓ 安全（权限缩小）
const int* p1 = &x;      // ✓

// const → 非 const: ✗ 危险（权限放大）
int* p2 = &cx;           // ✗ 编译错误
int* p3 = p1;           // ✗ 编译错误

// 需要强制转换（不推荐）
int* p4 = const_cast<int*>(&cx); // ⚠ 可以编译，但修改 cx 是未定义行为
```

四、const 与引用

4.1 常量引用（const 引用）

```
int x = 10;
const int& ref = x;    // ref 是 x 的常量引用

ref = 20;    // ✗ 不能通过 ref 修改 x
x = 20;      // ✓ 可以直接修改 x (ref 的值也变为 20)
```

注意：不存在 `int& const ref`（引用本身天然不可重新绑定，加 `const` 无意义）。

4.2 const 引用可以绑定到右值/临时对象

```
const int& r1 = 42;           // ✓ 绑定字面量
const int& r2 = x + y;        // ✓ 绑定表达式结果
const string& r3 = "hello";   // ✓ 绑定临时 string 对象

int& r4 = 42;                // ✗ 非 const 引用不能绑定右值
```

原理：编译器会创建一个临时对象，`const` 引用延长其生命周期。

```
const int& r = 42;
// 等价于：
// int __temp = 42;
// const int& r = __temp;
// __temp 的生命周期延长至 r 的作用域结束
```

4.3 const 引用的赋值兼容性

```
int x = 10;
const int cx = 20;

const int& r1 = x;    // ✓ 非const对象 → const引用（权限缩小）
const int& r2 = cx;    // ✓ const对象 → const引用
int& r3 = cx;         // ✗ const对象 → 非const引用（权限放大）

// 不同类型也可以（会产生临时对象）
double d = 3.14;
const int& r4 = d;    // ✓ 绑定到由 d 转换而来的临时 int 对象
int& r5 = d;         // ✗ 非const引用不能绑定临时对象
```

五、const 与函数

5.1 const 修饰函数参数

(1) const 值传递

```
void func(const int x) {
    x = 10; // ✗ 函数内不能修改 x
}

int a = 5;
func(a); // a 本身不受影响（值传递是拷贝）
```

对调用方无影响，只约束函数内部。实际开发中较少使用。

(2) const 引用传递

```
void print(const string& s) {
    // s 不可修改，避免了拷贝开销
    cout << s << endl;
}

const string cs = "hello";
string s = "world";

print(cs);           // ✓ const 对象可以传入
print(s);            // ✓ 非 const 对象也可以传入
print("temporary");  // ✓ 临时对象也可以传入
print(s + cs);       // ✓ 表达式结果（临时对象）也可以传入
```

不加 const 的问题：

```
void print(string& s) { // 非 const 引用
    cout << s << endl;
}

print(cs);           // ✗ const 对象不能传入
print("temporary");  // ✗ 临时对象不能传入
print(s + cs);       // ✗ 临时对象不能传入
```

(3) const 指针传递

```
void func(const int* p) {
    *p = 10; // ✗ 不能通过 p 修改所指内容
```

```

    p = nullptr; // ✓ 可以改变 p 本身 (p 是值传递)
}

void func2(int* const p) {
    *p = 10; // ✓ 可以修改所指内容
    p = nullptr; // ✗ 不能改变 p 本身
}

```

5.2 const 修饰函数返回值

返回 const 引用

```

class MyClass {
    string name;
public:
    // ✓ 返回成员的只读访问
    const string& getName() const {
        return name;
    }

    // ✓ 返回成员的可写访问
    string& getName() {
        return name;
    }
};

MyClass obj;
cout << obj.getName(); // 调用非 const 版本
const MyClass cobj;
cout << cobj.getName(); // 调用 const 版本
cobj.getName() = "test"; // ✗ 返回的是 const string&

```

返回 const 指针

```

class Array {
    int* data;
public:
    const int* getData() const {
        return data; // 调用方不能通过返回的指针修改数据
    }
};

```

六、const 成员函数（重点）

6.1 基本语法

```
class MyClass {
    int value;
public:
    int getValue() const {    // const 成员函数
        return value;
    }

    void setValue(int v) {    // 非 const 成员函数
        value = v;
    }
};
```

6.2 含义

const 成员函数中，this 指针变为 const MyClass* const this：

- 不能修改任何成员变量
- 不能调用其他非 const 成员函数

```
class MyClass {
    int value;
    int cache;
public:
    int compute() const {
        value = 10;    // ✗ 不能修改成员变量
        cache = 20;    // ✗ 不能修改成员变量
        setValue(10);   // ✗ 不能调用非 const 成员函数
        return value;   // ✓ 读取成员变量没问题
    }

    void setValue(int v) {
        value = v;      // ✓ 非 const 函数可以修改
    }
};
```

6.3 const 对象只能调用 const 成员函数

```
MyClass obj;
const MyClass cobj;
```

```
obj.getValue(); // ✓
obj.setValue(10); // ✓

cobj.getValue(); // ✓ const 对象调用 const 成员函数
cobj.setValue(10); // ✗ const 对象不能调用非 const 成员函数
```

6.4 const 重载

同一个函数可以有 const 和非 const 两个版本：

```
class Array {
    int data[10];
public:
    // 非 const 版本：返回可修改的引用
    int& operator[](int index) {
        cout << "non-const version" << endl;
        return data[index];
    }

    // const 版本：返回只读引用
    const int& operator[](int index) const {
        cout << "const version" << endl;
        return data[index];
    }
};

Array arr;
const Array carr;

arr[0] = 5; // 调用非 const 版本 ✓
int x = carr[0]; // 调用 const 版本 ✓
carr[0] = 5; // 调用 const 版本，返回 const int&, 不能赋值 ✗
```

非 const 对象优先调用非 const 函数；

const 对象只能调用 const 函数

6.5 mutable：突破 const 的限制

mutable 成员变量可以在 const 成员函数中被修改：

```
class MyClass {
    int value;
```

```

mutable int accessCount;    // mutable 修饰
mutable bool cacheValid;
mutable int cachedResult;
public:
    int getValue() const {
        ++accessCount;      // ✅ mutable 变量可以在 const 函数中修改

        if (!cacheValid) {
            cachedResult = /* 复杂计算 */;
            cacheValid = true; // ✅
        }
        return cachedResult;
    }
};

```

典型用途：

- 访问计数器
- 缓存/懒加载
- 互斥锁（`mutable std::mutex mtx;`）

七、const 与类

7.1 const 成员变量

```

class MyClass {
    const int id;        // const 成员变量
    const int maxSize;
public:
    // ✅ 必须在初始化列表中初始化
    MyClass(int i, int m) : id(i), maxSize(m) {
        // id = i;      // ❌ 不能在函数体中赋值
    }
};

```

7.2 const 对象

```

const MyClass obj(1, 100);
// obj 的所有成员都视为 const
// 只能调用 const 成员函数

```

7.3 const 与构造/析构函数


```
class MyClass {
public:
    MyClass() {}           // 构造函数不能是 const (需要初始化成员)
    ~MyClass() {}          // 析构函数不能是 const (需要清理资源)
    // const MyClass() {}   // ✗ 语法错误
};
```

7.4 static 与 const

```
class MyClass {
    static const int SIZE = 100;           // ✓ 类内初始化 (整型)
    static constexpr double PI = 3.14;    // ✓ C++11 constexpr
    static const string NAME;              // 声明
};

// 类外定义 (非整型的 static const 需要类外定义)
const string MyClass::NAME = "hello";
```

八、const 在运算符重载中的应用

8.1 各种运算符的 const 规范

```
class MyClass {
public:
    // 算术运算符: 产生新值, 不修改操作数
    MyClass operator+(const MyClass& rhs) const;
    MyClass operator-(const MyClass& rhs) const;
    MyClass operator*(const MyClass& rhs) const;

    // 复合赋值运算符: 修改自身, 返回自身引用
    MyClass& operator+=(const MyClass& rhs);    // 不加 const! 需要修改 *this
    MyClass& operator-=(const MyClass& rhs);

    // 赋值运算符: 修改自身
    MyClass& operator=(const MyClass& rhs);      // 不加 const!

    // 比较运算符: 不修改任何操作数
    bool operator==(const MyClass& rhs) const;
    bool operator<(const MyClass& rhs) const;
```

```
// 下标运算符: const 和非 const 两个版本
int& operator[](int index);
const int& operator[](int index) const;

// 前置++: 修改自身
MyClass& operator++();           // 不加 const
// 后置++: 返回修改前的副本
MyClass operator++(int);        // 不加 const (需要修改自身)

// 输出运算符 (友元)
friend ostream& operator<<(ostream& os, const MyClass& obj);
//不修改 obj
};
```